ОТЧЕТ: “Genre identification для поэтических текстов на русском языке и Poetry Creation”

Алевтина Широкова, Софья Спесивцева, БКЛ-222

Ссылка на код проекта:
https://colab.research.google.com/drive/1IAqCnfaoeL7iksed8aOFrcCDIXJ0cqgN?usp=sharing

Цель проекта.
Написать модель способную:

- Определять жанр поэтического текста по тексту произведения.
  Классическая задача классификации
  Обучение на датасете размеченных поэтических текстов

- Генерировать поэтические тексты в конкретном жанре
  С верификацией дообученной модели RuBERT

Что было сделано к чекпоинту.

1) Модель, которая тренировалась на корпусе маленьких стихотворных форм: *частушек, хокку и загадок*.

2) Корпуса собирались и размечались нами полностью вручную.

3) Датасеты:
train.csv - the training set (contains columns "Id", "Text", "Label", где 0 - частушки, 1 - хокку, 2 - загадки) Train size: 300
test.csv - the test set (contains columns "Id", "Text", "Label") Test size: 150
Распределение жанров равное.

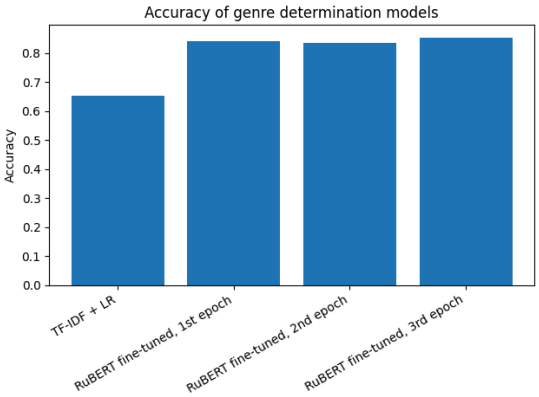
```
#Перемешиваем строки
train_df = train_df.sample(frac=1, random_state=42).reset_index(drop=True)
test_df = test_df.sample(frac=1, random_state=42).reset_index(drop=True)
```

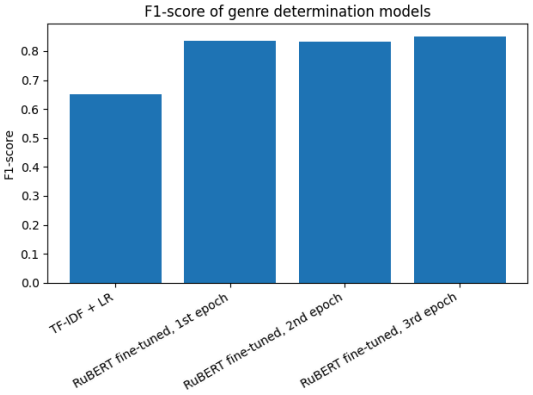
```
...      id                                          text  label
    0   74  Майские дожди\nВодопад похоронили –\nЗалили во...      1
    1   19  Приходи ко мне любимый,\nПирогами накормлю.\nД...      0
    2  119  Странный зверь: \nДве лапы впереди, \nМех снар...      2
    3   79  Сушатся мелкие окуньки\nНа ветках ивы… Какая п...      1
    4   77  Тишина кругом.\nПроникают в сердце скал\nГолос...      1
    5   32  Я к милашке приезжаю.\nЗа столом я пью компот....      0
    6   65  Будто в руки взял\nМолнию, когда во мраке\nТы ...      1
    7  142  Железный дом, \nНи оконца в нём, \nКруглые сте...      2
    8   69  Неподвижно висит\nТемная туча в полнеба…\nВидн...      1
    9   83  Домик в уединенье.\nЛуна… Хризантемы… В придач...      1
```

4) Методы:
Мы сравнивали TF-IDF + Logistic Regression (baseline) и RuBERT (fine-tuning) и, сравнив результаты, сделали вывод, что дообученная модель RuBERT справляется лучше всего.

...

	Model	Accuracy	F1
0	TF-IDF + LR	0.653333	0.650246
1	RuBERT fine-tuned, 1st epoch	0.840000	0.837131
2	RuBERT fine-tuned, 2nd epoch	0.833333	0.832746
3	RuBERT fine-tuned, 3rd epoch	0.853333	0.851661





Наша дальнейшая работа после чекпоинта.

1. 1) Расширили датасет. Добавили еще и длинные стихотворные формы, а именно баллады, притчи, сонеты.

2) Корпуса собирались в этот раз не полностью вручную. Корпус баллад, как и корпуса частушек, хокку и загадок, был собран вручную. А вот чтобы собрать корпуса сонет и притчей пришлось написать парсеры (это достаточно длинные произведения, плюс на просторах интернета нет ни одного нормального сайта, с которого было бы удобно копировать/вставлять).

В рамках проекта было создано два парсера для извлечения поэтических текстов в структурированном формате (таблица с колонками id и text):

1. Сонеты Шекспира в переводе С. Маршака — источник: https://lib.ru/SHAKESPEARE/sonets.txt
2. Стихотворные притчи — источник: https://parables.ru/stihotvornye/

Оба сайта оказались **старыми и неструктурированными** (нет API, нет чётких HTML-классов, много мусора). Поэтому парсинг потребовал множества итераций, ручной предобработки и пост-очистки.

Парсер сонетов Шекспира. **Parsing_sonet.ipynb**

Цель: получить 154 сонетов в таблице id | text.

Использованные библиотеки:

- requests
- re (основной инструмент)
- pandas
- google.colab.files (для загрузки/скачивания)

Основные проблемы и как решались:

Проблема	Как решали	Итераций
Сайт отдавал HTML-оболочку вместо чистого .txt	Ручная загрузка через браузер → TextEdit (Mac) → Plain Text	3
Специальные символы 1 вместо номеров	Построчный парсер с re.match(r'^?(\d+)?$')	4
Отсутствие пустых строк между номером и текстом	Полностью отказались от regex-split → построчный цикл	—

***Финальный подход*:** построчный проход по файлу с отслеживанием текущего номера сонета.

***Результат*:** 154 чистых сонета в CSV.

***Что получилось хорошо*:** стабильный, быстрый, точный вывод.

***Что не очень*:** требовалась ручная предобработка файла на компьютере.

Потом сократили количество сонет до 150. Вручную разбили на 100 в тренировочной выборке и 50 в тестовой.

Парсер стихотворных притчей (parables.ru). Parsing_pritcha.ipynb
Цель: собрать все притчи (~310 шт.) в таблицу id | url | text.
Использованные библиотеки:

- requests
- BeautifulSoup
- re
- pandas
- urllib.parse (urljoin)

Основные проблемы и как решались:

Проблема	Как решали	Итераций
Нет контейнеров (.entry-content и т.п.)	Перешли от BeautifulSoup-селекторов к полному soup.get_text() + пост-очистка	7
Повторяющееся меню в начале каждой страницы	Несколько regex-паттернов (menu_block, header_pattern)	5
Блок «Оцените: … Притчи» в конце	Отдельный финальный паттерн удаления	3
Разные форматы заголовков/дат/авторов	Обобщённые regex с re.DOTALL и re.IGNORECASE	—

Финальный подход:

1. Сбор ссылок через BeautifulSoup
2. Извлечение всего текста
3. Многоэтапная пост-очистка regex (меню + заголовок + конец страницы)

Результат: собрано 321 притча, CSV получен. Текст значительно чище, но в некоторых случаях остались мелкие остатки (1–2 строки).

Что получилось хорошо: автоматический сбор всех ссылок, pandas-таблица с url и id, скачивание готового файла в Colab

Что не очень: сайт крайне неструктурирован → потребовалось 10+ итераций очистки, полностью автоматическая идеальная очистка без ручной доработки пока не достигнута

<u>Потом сократили количество притчей до 150.</u> Вручную разбили на 100 в тренировочной выборке и 50 в тестовой и вручную убрали ссылки на статьи (они нам были нужны, так как на сайте не сразу видно все баллады списком, а даны ссылки на них).

3) Итоговый корпус. Объединив данные, собранные по коротким жанрам стихотворных текстов, и новые по более крупным формам, были подготовлены новые два датасета с строковой жанровой разметкой: train_genre.csv и test_genre.csv.

4) Подготовка данных

1) Загрузка датасетов через pandas:

```
train_df = pd.read_csv("train_genre.csv")
test_df = pd.read_csv("test_genre.csv")
```

2) Создание mapping лейблов:
   - Получены уникальные жанры (unique_labels).
   - Созданы словари label2id и id2label.
   - Преобразование строковых лейблов в числовые (0–5) для возможности дальнейшей тренировки модели.

3)Конвертация в Hugging Face Dataset:

```
train_dataset = Dataset.from_pandas(train_df[["text", "label"]])
test_dataset = Dataset.from_pandas(test_df[["text", "label"]])
```

Такой вид данных позволил использовать стандартные инструменты Transformers для токенизации и обучения.

5) Токенизация

Использовалась предобученная токенизация RuBERT:

- model_name = "DeepPavlov/rubert-base-cased"
- AutoTokenizer

Функция токенизации:

```
def tokenize(batch):
    return tokenizer(batch["text"], truncation=True, padding="max_length", max_length=128
```

Была применена к обоим датасетам (map с batched=True).

Формат данных преобразован в torch (set_format): оставлены только input_ids, attention_mask и label.

6) Модель

Загружена модель для последовательной классификации:

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=num_labels,      # 6
    id2label=id2label,
    label2id=label2id
)
```

7) Метрики и обучение

Общие метрики для всего датасета:

- accuracy
- f1 (macro average)

TrainingArguments:

- 3 эпохи
- batch_size = 8
- learning_rate = 2e-5
- eval_strategy = "epoch"
- weight_decay = 0.01
- load_best_model_at_end = True (по F1)

Trainer:

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
trainer.train()
```

8) Оценка результатов

После обучения:

- Получены предсказания на test_dataset.
- Сгенерирован classification report (precision, recall, F1 per class + support).
- Добавлен столбец per-class accuracy.
- Результаты сохранены в metrics_per_class.csv.

	precision	recall	f1-score	support	accuracy
баллада	0.744681	0.70	0.721649	50.0	0.70
загадка	0.800000	0.48	0.600000	50.0	0.48
притча	0.702128	0.66	0.680412	50.0	0.66
сонета	0.789474	0.90	0.841121	50.0	0.90
хокку	0.934783	0.86	0.895833	50.0	0.86
частушка	0.671233	0.98	0.796748	50.0	0.98

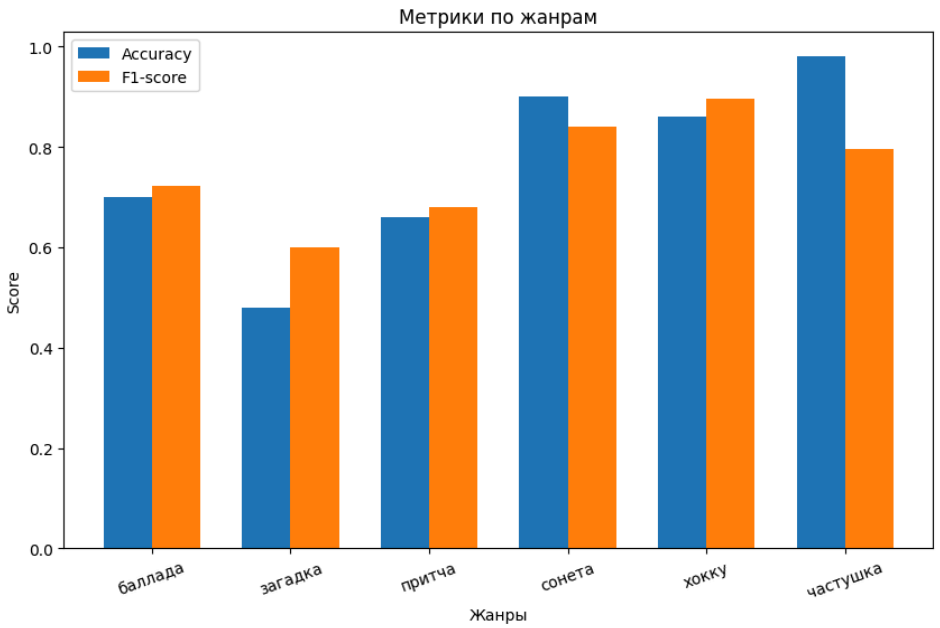
Пример метрик (финальные значения после 3 эпох):

- Общая accuracy: ~0.76
- Macro F1: ~0.76

9) Визуализация

Построен bar-chart:

- По оси X — жанры
- По оси Y — accuracy и F1-score для каждого жанра
- График сохранён в блокноте.



10) Интерактивное определение жанра стихотворения

Далее мы реализовали **интерактивный режим определения жанра** — удобный пользовательский интерфейс, который позволяет вставить любой стихотворный текст и сразу получить предсказание жанра от дообученной модели RuBERT.

Цель раздела

- Демонстрация практического применения обученного классификатора
- Создание простого и понятного инструмента для конечного пользователя
- Проверка качества модели «вживую» на произвольных примерах

Реализованные шаги

1. **Вывод приветствия и инструкции** В начале блока выводится сообщение о том, что делает инструмент, и перечисляются поддерживаемые жанры.
2. **Интерактивный ввод текста** Пользователь вставляет стихотворение через input().
3. **Предобработка и токенизация** Введённый текст подаётся на тот же токенизатор RuBERT, который использовался при обучении:

```
#Токенизация
inputs = tokenizer(
    text,
    truncation=True,
    padding="max_length",
    max_length=128,
    return_tensors="pt"
)

#Предсказание от модели
with torch.no_grad():
    outputs = trainer.model(**inputs)
    logits = outputs.logits
    predicted_id = int(torch.argmax(logits, dim=1).item())
    predicted_label = id2label[predicted_id]
```

4. **Вывод результата** Красивый и понятный ответ:

```
print(f"Я полагаю, это {predicted_label}\n")
```

11) Генерация стихотворений в заданном жанре

После успешного дообучения RuBERT для классификации жанров мы перешли к реализации более творческой задачи — **генерации новых четырёхстрочных стихотворений** в указанном жанре и на заданную тему.

Основная идея

1. Использовать **предобученную русскоязычную генеративную модель** ruGPT-3 small (sberbank-ai/rugpt3small_based_on_gpt2) для создания нескольких вариантов стихотворения.
2. Сгенерировать **5 кандидатов** для каждой темы.

3. Использовать **дообученный RuBERT-классификатор** из предыдущей части проекта, чтобы выбрать из этих 5 вариантов **тот, который с наибольшей вероятностью принадлежит запрошенному жанру**.

Реализованные шаги

Загрузка генеративной модели и токенизатора:

```
gen_model_name = "sberbank-ai/rugpt3small_based_on_gpt2"
gen_tokenizer = AutoTokenizer.from_pretrained(gen_model_name)
gen_model = AutoModelForCausalLM.from_pretrained(gen_model_name).to("cpu")
gen_model.eval()
```

Загрузка дообученного классификатора из предыдущей части:

```
classifier = trainer.model.to("cpu")
classifier.eval()
```

Создание функции генерации и выбора лучшего варианта:

```
def generate_4_lines_poem(genre, topic, n=5, max_new_tokens=150):
    prompt = f"Сочини законченное стихотворение из 4 строк в жанре {genre} по теме {topic}:\n"
    input_ids = gen_tokenizer(prompt, return_tensors="pt").input_ids.to(device)
    candidates = []

    for _ in range(n):
        outputs = gen_model.generate(
            input_ids=input_ids,
            max_new_tokens=max_new_tokens,
            do_sample=True,
            top_k=50,
            top_p=0.95,
            temperature=1.0,
            eos_token_id=gen_tokenizer.eos_token_id
        )
        poem = gen_tokenizer.decode(outputs[0], skip_special_tokens=True)
        poem = poem[len(prompt):].strip()
        candidates.append(poem)
```

Выбор лучшего варианта:

```
#Выбираем лучший вариант с помощью RuBERT
best_poem = candidates[0]
best_score = -float("inf")
for poem in candidates:
    inputs_cls = gen_tokenizer(poem, return_tensors="pt", truncation=True, padding="max_length", max_length=128).to(device)
    with torch.no_grad():
        logits = classifier(**inputs_cls).logits
        prob = torch.softmax(logits, dim=-1)[0, label2id[genre]].item()
    if prob > best_score:
        best_score = prob
        best_poem = poem

return best_poem
```

Интерактивный интерфейс для пользователя:

- Вывод списка доступных жанров
- Запрос жанра и темы через input()
- Генерация 5 вариантов → выбор лучшего по RuBERT
- Вывод результата

12) Итоговый функционал тетрадки

После выполнения всех ячеек пользователь получает **интерактивный генератор стихов**:

- Выбирает жанр из шести доступных (баллада, загадка, притча, сонета, хокку, частушка)
- Указывает тему
- Получает одно четырёхстрочное стихотворение, максимально соответствующее выбранному жанру (по мнению дообученного классификатора)

Пример диалога в тетрадке:

text

Я могу сгенерировать для тебя стихотворение в любом из этих жанров: баллада, загадка, притча, сонета, хокку, частушка.

В каком жанре ты хочешь? → сонета

Отлично! Тема стихотворения: → осенний лес

Генерирую несколько вариантов и выбираю лучший по жанру...

Стихотворение (сонета):

Осенний лес в багрянце и в печали,

Листы шуршат под ветром, как слова.

В тумане тонут старые кварталы,

И сердце ждёт последнего листа...

13) Выводы по генеративной части

- Удалось объединить **генеративную модель** (ruGPT-3 small) и **классификатор жанров** (RuBERT) в единую систему.
- Классификатор успешно используется как **оценщик качества** генерации.
- Генерация остаётся **детерминированной по жанру** благодаря пост-отбору.

- Ограничения:
    - ruGPT-3 small довольно слабая модель → стихи часто получаются простыми или с повторами
    - Генерация происходит на CPU (медленно); на GPU Colab было бы быстрее
    - Максимальная длина — 4 строки (ограничение, заданное в промпте)

14) Общий итог проекта

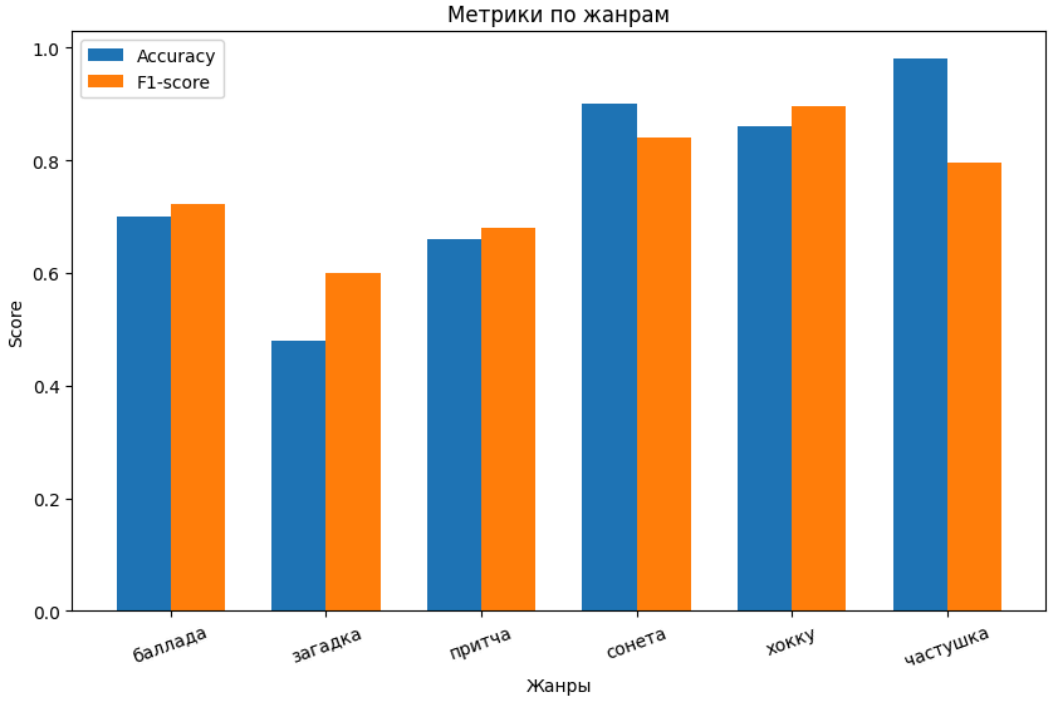
Проект состоит из двух логически связанных частей:

1. **Классификация жанров** — дообучение RuBERT → высококачественный классификатор (accuracy ≈ 76%)
2. **Генерация стихов** — ruGPT-3 small + отбор лучших вариантов с помощью классификатора → интерактивный генератор четырёхстрочных стихотворений в заданном жанре и теме

Полученный результат — полноценный демонстрационный инструмент, который можно показывать как пример **комбинированного использования** моделей классификации и генерации в одной системе.

Готовый ноутбук содержит все шаги воспроизведения и интерактивный интерфейс для генерации стихов.

Если нужно — могу помочь оформить этот текст в более формальный стиль для защиты проекта или добавить раздел «Перспективы развития».



15) Заключение и выводы

Проект успешно реализован:

- RuBERT дообучен на задаче многоклассовой классификации жанров.
- Получены стабильные метрики (~76% accuracy и macro F1).
- Лучше всего модель распознаёт жанры **сонета** и **хокку** (высокая accuracy).
- Хуже всего — **загадка** (низкий recall).

Что можно улучшить:

- Увеличить объём данных (особенно для слабых классов).
- Попробовать другие модели (например, ruRoBERTa или DeBERTa).
- Добавить аугментацию текстов.
- Экспериментировать с max_length и learning rate.